

JavaScript

JavaScript Introduction

• **Question 1: What is JavaScript? Explain the role of JavaScript in web development.**

Ans:

JavaScript is a high-level, lightweight, interpreted programming language that is mainly used to create interactive and dynamic web pages. It was developed to make websites more responsive and user-friendly. Earlier, websites were mostly static and could only display fixed content using HTML and CSS. JavaScript added the ability to perform actions, update content dynamically, and respond to user interactions without reloading the page.

JavaScript is one of the core technologies of web development along with:

- HTML – used for webpage structure
- CSS – used for styling and design
- JavaScript – used for functionality and interactivity

Role of JavaScript in Web Development

JavaScript plays a very important role in modern web development because it controls the behavior of web pages. It allows developers to create interactive features and improve user experience.

Main Roles of JavaScript:

1. **Client-Side Scripting**
 - JavaScript runs directly in the user's browser.
 - It reduces server load and improves website speed.
2. **Dynamic Content**
 - It can change webpage content without refreshing the page.
 - Example: live notifications, chat applications, and updating product prices.
3. **User Interaction**
 - JavaScript responds to user actions such as:
 - button clicks
 - mouse movements

- keyboard input
- form submissions
- 4. **Form Validation**
 - It checks user input before sending data to the server.
 - Example: checking if an email field is empty or password length is correct.
- 5. **Animations and Effects**
 - JavaScript is used to create sliders, image galleries, popups, and animations.
- 6. **API Communication**
 - JavaScript can fetch data from servers using APIs.
 - Example: weather apps and live sports scores.
- 7. **Modern Application Development**
 - JavaScript is used in frontend frameworks such as React, Angular, and Vue.js.
 - Backend development is also possible using Node.js.

Thus, JavaScript has become one of the most important programming languages for web development.

• Question 2: How is JavaScript different from other programming languages like Python or Java?

Ans:

JavaScript is different from Python and Java in many ways such as purpose, execution, syntax, and usage area. Although all three are programming languages, they are designed for different types of applications and environments.

JavaScript is mainly used for web development and runs directly inside web browsers. It is responsible for making websites interactive and dynamic. Features such as animations, form validation, live updates, dropdown menus, and interactive buttons are commonly created using JavaScript. It is widely used in frontend development and can also be used for backend development through technologies like Node.js.

Python, on the other hand, is a general-purpose programming language known for its simple and readable syntax. It is widely used in areas such as artificial intelligence, machine learning, data science, automation, and software development. Python is considered beginner-friendly because its syntax is easier to understand compared to many other languages.

Java is a strongly typed and object-oriented programming language mainly used for enterprise-level applications, Android development, desktop software, and large

systems. Java follows strict programming rules and requires developers to declare data types clearly. It is known for its security, portability, and performance.

One major difference between JavaScript and Java is that JavaScript runs directly in web browsers, while Java programs run on the Java Virtual Machine (JVM). Python programs run using a Python interpreter. JavaScript is interpreted and supports event-driven and asynchronous programming, making it highly suitable for modern web applications.

Another important difference is syntax and typing system. JavaScript and Python are dynamically typed languages, meaning variable types are decided during runtime. Java is statically typed, which means variable types must be declared before execution. Because of this, Java code is usually longer and stricter compared to JavaScript and Python.

In terms of usage, JavaScript is mainly focused on web technologies, Python is focused on simplicity and versatile programming tasks, and Java is focused on secure and scalable application development.

Therefore, while JavaScript, Python, and Java are all powerful programming languages, JavaScript is specially designed for creating interactive web applications and browser-based functionality.

• Question 3: Discuss the use of <script> tag in HTML. How can you link an external JavaScript file to an HTML document?

Ans:

The <script> tag in HTML is used to include JavaScript code in a webpage. It allows the browser to execute JavaScript instructions and add functionality to the webpage.

Purpose of the <script> Tag

The <script> tag is used for:

- Adding interactivity
- Executing JavaScript code
- Linking external JavaScript files
- Manipulating webpage content dynamically
- Communicating with servers through APIs

Placement of <script> Tag

The <script> tag can be placed:

- Inside the <head> section
- Inside the <body> section

Best Practice

Usually, developers place the <script> tag before the closing </body> tag because:

- HTML content loads first
- Improves webpage performance
- Prevents page rendering delays

External JavaScript

- In external JavaScript, the JavaScript code is stored in a separate .js file and linked to the HTML document using the src attribute

- **Step 1: Create JavaScript File**
- **script.js**

- ```
function greet() {
 alert("Welcome to JavaScript");
}
```

- **Step 2: Link File with HTML**

- ```
<!DOCTYPE html>  
<html>  
<head>  
    <title>External JavaScript</title>  
</head>  
<body>  
  
    <button onclick="greet()">Click Here</button>  
  
    <script src="script.js"></script>  
  
</body>  
</html>
```

Variables and Data Types

• Question 1: What are variables in JavaScript? How do you declare a variable using var, let and const?

Ans:

In JavaScript, variables are used to store data or values that can be used and manipulated during program execution. A variable acts like a container that holds information such as numbers, text, or boolean values. Variables make programming flexible because stored values can be reused throughout the program.

JavaScript provides three keywords to declare variables:

- var
- let
- const

These keywords differ in scope, redeclaration rules, and mutability.

1. var Keyword

The `var` keyword was used in older versions of JavaScript to declare variables. Variables declared with `var` are function-scoped, meaning they are accessible within the function where they are created.

Characteristics of `var`

- Can be redeclared
- Can be updated
- Function-scoped
- Supports hoisting

Example:

```
var name = "Aryan";  
var age = 21;
```

```
age = 22;
```

Although `var` is still supported, modern JavaScript development usually prefers `let` and `const` because they provide better scope management.

2. let Keyword

The `let` keyword was introduced in ES6 (ECMAScript 2015). It is used to declare block-scoped variables.

Characteristics of `let`

- Cannot be redeclared in the same scope
- Can be updated
- Block-scoped
- Safer than `var`

Example:

```
let city = "Surat";
```

```
city = "Rajkot";
```

`let` is commonly used when the variable value may change later in the program.

3. const Keyword

The `const` keyword is also introduced in ES6 and is used to declare constant variables.

Characteristics of `const`

- Cannot be redeclared
- Cannot be updated
- Block-scoped
- Must be initialized during declaration

Example:

```
const country = "India";
```

If you try to change the value of a `const` variable, JavaScript will generate an error.

`const` is generally used for values that should remain fixed throughout the program.

• Question 2: Explain the different data types in JavaScript. Provide examples for each.

Ans:

In JavaScript, data types define the type of value that a variable can store. JavaScript is a dynamically typed language, which means the data type is automatically determined during runtime.

JavaScript data types are mainly divided into:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

1. Primitive Data Types

Primitive data types store single values and are immutable.

a) Number

The `Number` data type is used to store numeric values, including integers and decimal numbers.

Example:

```
let age = 21;  
let price = 99.99;
```

b) String

The `String` data type is used to store text or characters enclosed in quotes.

Example:

```
let name = "abcd";
```

c) Boolean

The Boolean data type stores only two values:

- true
- false

Example:

```
let isLoggedIn = true;
```

Booleans are mainly used in conditions and comparisons.

d) Undefined

A variable that is declared but not assigned a value automatically gets the value undefined.

Example:

```
let data;
```

e) Null

null represents an intentional empty value or absence of data.

Example:

```
let result = null;
```

f) BigInt

BigInt is used to store very large integers beyond the safe limit of normal numbers.

Example:

```
let bigNumber = 12345678901234567890n;
```

2. Non-Primitive Data Type

Object

Objects are collections of key-value pairs and are used to store complex data.

Example:

```
let student = {  
  name: "Aryan",  
  age: 21  
};
```

Arrays and functions are also considered objects in JavaScript.

• Question 3: What is the difference between undefined and null in JavaScript?

Ans:

In JavaScript, both undefined and null represent absence of value, but they are different in meaning and usage.

undefined means a variable has been declared but has not yet been assigned any value. It is automatically assigned by JavaScript.

Example:

```
let name;  
console.log(name);
```

Output:

undefined

On the other hand, null is an intentional assignment that represents an empty or unknown value. It is manually assigned by the programmer.

Example:

```
let data = null;
```

JavaScript Operators

• **Question 1: What are the different types of operators in JavaScript? Explain with examples.**

- **Arithmetic operators**
- **Assignment operators**
- **Comparison operators**
- **Logical operators**

Ans:

In JavaScript, operators are special symbols or keywords used to perform operations on values and variables. Operators are used in calculations, comparisons, logical conditions, and assigning values.

JavaScript provides different types of operators, among which the most commonly used are:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication, and division.

Types of Arithmetic Operators

Operator	Meaning	Example
----------	---------	---------

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (Remainder)	a % b
++	Increment	a++
--	Decrement	a--

Example:

```
let a = 10;  
let b = 5;  
  
console.log(a + b); // 15  
console.log(a - b); // 5  
console.log(a * b); // 50  
console.log(a / b); // 2  
console.log(a % b); // 0
```

Arithmetic operators are mainly used in calculations and mathematical expressions.

2. Assignment Operators

Assignment operators are used to assign values to variables.

Types of Assignment Operators

Operator	Meaning	Example
=	Assign value	x = 10
+=	Add and assign	x += 5
-=	Subtract and assign	x -= 5

Operator	Meaning	Example
----------	---------	---------

<code>*=</code>	Multiply and assign	<code>x *= 2</code>
-----------------	---------------------	---------------------

<code>/=</code>	Divide and assign	<code>x /= 2</code>
-----------------	-------------------	---------------------

Example:

```
let x = 10;
```

```
x += 5;  
console.log(x); // 15
```

```
x -= 3;  
console.log(x); // 12
```

Assignment operators reduce the need to write repeated code.

3. Comparison Operators

Comparison operators are used to compare two values. They return either true or false.

Types of Comparison Operators

Operator	Meaning
----------	---------

<code>==</code>	Equal to
-----------------	----------

<code>===</code>	Strict equal to
------------------	-----------------

<code>!=</code>	Not equal
-----------------	-----------

<code>!==</code>	Strict not equal
------------------	------------------

<code>></code>	Greater than
-------------------	--------------

<code><</code>	Less than
-------------------	-----------

<code>>=</code>	Greater than or equal
--------------------	-----------------------

<code><=</code>	Less than or equal
--------------------	--------------------

Example:

```
let a = 10;  
let b = "10";  
  
console.log(a == b); // true  
console.log(a === b); // false  
console.log(a > 5); // true
```

Comparison operators are mainly used in conditions and decision-making statements.

4. Logical Operators

Logical operators are used to combine or evaluate multiple conditions.

Types of Logical Operators

Operator	Meaning
----------	---------

&&	Logical AND
----	-------------

	Logical OR
--	------------

!	Logical NOT
---	-------------

Example:

```
let age = 20;  
let hasID = true;  
  
console.log(age > 18 && hasID); // true  
console.log(age < 18 || hasID); // true  
console.log(!hasID);           // false
```

Explanation

- && returns true only if both conditions are true.
- || returns true if at least one condition is true.
- ! reverses the result.

Logical operators are widely used in authentication systems, validations, and conditional statements.

• Question 2: What is the difference between == and === in JavaScript?

Ans:

In JavaScript, both == and === are comparison operators used to compare values, but they work differently.

1. Double Equal (==)

The == operator is called the equality operator. It compares only the values and automatically converts data types if needed. This process is known as type conversion or type coercion.

Example:

```
let a = 10;  
let b = "10";  
  
console.log(a == b);
```

Output:

true

Explanation

Although a is a number and b is a string, JavaScript converts the string "10" into the number 10, so both values become equal.

2. Triple Equal (===)

The === operator is called the strict equality operator. It compares both:

- value
- data type

No type conversion takes place.

Example:

```
let a = 10;  
let b = "10";  
  
console.log(a === b);
```

Output:

false

Explanation

The values may look similar, but their data types are different:

- a → Number
- b → String

Therefore, === returns false.

Control Flow (If-Else, Switch)

**• Question 1: What is control flow in JavaScript?
Explain how if-else statements work with an example.**

Ans:

In JavaScript, control flow refers to the order in which statements and instructions are executed in a program. Normally, JavaScript executes code from top to bottom, but sometimes decisions need to be made based on conditions. Control flow statements help the program decide which block of code should run and which should be skipped.

Control flow is important because it allows programs to:

- make decisions
- repeat tasks
- handle different situations dynamically

Some common control flow statements in JavaScript are:

- if
- if-else
- else if
- switch
- loops (for, while)

if-else Statement in JavaScript

The if-else statement is used for decision making. It checks whether a condition is true or false.

- If the condition is true, the code inside the if block executes.
- If the condition is false, the code inside the else block executes.

Syntax of if-else

```
if (condition) {  
    // code executes if condition is true  
} else {  
    // code executes if condition is false  
}
```

Example of if-else

```
let age = 20;  
  
if (age >= 18) {  
    console.log("Eligible for voting");  
} else {  
    console.log("Not eligible for voting");  
}
```

Output:

Eligible for voting

• Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

Ans:

In JavaScript, the switch statement is a control flow statement used to execute different blocks of code based on different conditions or values. It is mainly used when a variable or expression may have multiple possible values.

The switch statement compares a single value against multiple cases. When a matching case is found, the corresponding block of code is executed.

Syntax of switch Statement

```
switch(expression) {  
  
    case value1:  
        // code block  
        break;  
  
    case value2:  
        // code block  
        break;  
  
    default:  
        // default code  
}
```

How switch Works

1. The expression inside switch() is evaluated.
2. JavaScript compares the result with each case.
3. If a matching case is found, that block executes.
4. The break statement stops further execution.
5. If no case matches, the default block executes.

Example of switch Statement

```
let day = 3;  
  
switch(day) {  
  
    case 1:  
        console.log("Monday");  
        break;
```

```
case 2:
  console.log("Tuesday");
  break;

case 3:
  console.log("Wednesday");
  break;

default:
  console.log("Invalid Day");
}
```

Output:

Wednesday

Role of break Statement

The break statement is very important in a switch statement because it stops execution after the matching case.

Without break, JavaScript continues executing the next cases. This behavior is called fall-through.

default Case

The default block executes when none of the cases match the expression.

It works similarly to the else block in an if-else statement.

When to Use switch Instead of if-else

A switch statement is preferred over if-else when:

- multiple conditions depend on the same variable
- there are many possible fixed values
- code readability needs improvement

Example Situation

Using switch is better for:

- menu selection systems
- calculator operations
- day or month selection
- user role management

Loops (For, While, Do-While)

• **Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.**

Ans:

In JavaScript, loops are control flow statements used to execute a block of code repeatedly until a specified condition becomes false. Loops help reduce code repetition and make programs more efficient.

JavaScript mainly provides three types of loops:

1. for loop
2. while loop
3. do-while loop

Loops are commonly used for:

- printing values
- processing arrays
- repeating tasks
- performing calculations
- data iteration

1. for Loop

The for loop is used when the number of iterations is known in advance. It is one of the most commonly used loops in JavaScript.

A for loop consists of three parts:

1. Initialization
2. Condition
3. Increment/Decrement

Syntax of for Loop

```
for(initialization; condition; increment/decrement) {  
    // code block  
}
```

Example of for Loop

```
for(let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

Explanation

- `let i = 1` initializes the variable.
- `i <= 5` checks the condition.
- `i++` increases the value after every iteration.
- The loop stops when the condition becomes false.

2. while Loop

The while loop executes a block of code as long as the condition remains true. It is useful when the number of iterations is not known beforehand.

Syntax of while Loop

```
while(condition) {  
    // code block  
}
```

Example of while Loop

```
let i = 1;  
  
while(i <= 5) {  
    console.log(i);  
    i++;  
}
```

Output:

```
1
2
3
4
5
```

Explanation

- The condition is checked before execution.
- If the condition is true, the loop runs.
- If the condition becomes false, the loop stops.

3. do-while Loop

The do-while loop is similar to the while loop, but it executes the code block at least once before checking the condition.

Syntax of do-while Loop

```
do {
    // code block
} while(condition);
```

Example of do-while Loop

```
let i = 1;
```

```
do {
    console.log(i);
    i++;
} while(i <= 5);
```

Output:

```
1
2
3
4
5
```

Explanation

- The code inside do executes first.
- After execution, the condition is checked.
- If true, the loop continues.
- If false, the loop stops.

• Question 2: What is the difference between a while loop and a do-while loop?

Ans:

In JavaScript, both while and do-while loops are used for repeated execution of code blocks based on conditions. However, the main difference lies in when the condition is checked.

while Loop

In a while loop, the condition is checked before executing the loop body. If the condition is false initially, the loop will not execute even once.

Syntax:

```
while(condition) {  
    // code block  
}
```

Example:

```
let i = 6;  
  
while(i <= 5) {  
    console.log(i);  
    i++;  
}
```

Output:

No output

Explanation

The condition `i <= 5` is false from the beginning, so the loop body never executes.

do-while Loop

In a do-while loop, the code block executes first, and then the condition is checked. Therefore, the loop always executes at least one time, even if the condition is false initially.

Syntax:

```
do {  
    // code block  
} while(condition);
```

Example:

```
let i = 6;  
  
do {  
    console.log(i);  
    i++;  
} while(i <= 5);
```

Output:

6

Explanation

The loop executes once before checking the condition, so 6 is printed even though the condition is false.

Functions

• **Question 1: What are functions in JavaScript?**
Explain the syntax for declaring and calling a function.

Ans:

In JavaScript, a function is a reusable block of code designed to perform a specific task. Functions help organize code into smaller, manageable parts and reduce repetition in programs.

Instead of writing the same code multiple times, a function can be created once and called whenever needed. Functions are one of the most important concepts in JavaScript because they improve code readability, reusability, and maintainability.

Functions are widely used for:

- calculations
- form validation
- event handling
- data processing
- API requests
- creating reusable logic

Syntax for Declaring a Function

A function is declared using the `function` keyword followed by:

1. Function name
2. Parentheses `()`
3. Curly braces `{ }`

Basic Syntax

```
function functionName() {  
    // code block  
}
```

Example of Function Declaration

```
function greet() {  
    console.log("Welcome to JavaScript");  
}
```

In this example:

- `function` is the keyword used to create a function.
- `greet` is the function name.
- The code inside `{ }` is the function body.

Calling a Function

Declaring a function only creates it. To execute the function, it must be called using its name followed by parentheses.

Example:

```
greet();
```

Output:

Welcome to JavaScript

When the function is called, the code inside the function body executes.

• Question 2: What is the difference between a function declaration and a function expression?

Ans:

In JavaScript, functions can be created in different ways. Two common methods are:

1. Function Declaration
2. Function Expression

Although both are used to create functions, they differ in syntax, behavior, and hoisting.

1. Function Declaration

A function declaration defines a named function using the `function` keyword.

Syntax:

```
function greet() {  
  console.log("Hello");  
}
```

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(5, 3));
```

Output:

8

Characteristics of Function Declaration

- Functions are declared with a name.
- They are hoisted completely.
- The function can be called before declaration.

Example:

```
hello();  
  
function hello() {  
  console.log("Hi");  
}
```

This works because function declarations are hoisted in JavaScript.

2. Function Expression

A function expression stores a function inside a variable.

Syntax:

```
const greet = function() {  
  console.log("Hello");  
};
```

Example:

```
const multiply = function(a, b) {  
  return a * b;  
};  
  
console.log(multiply(4, 5));
```

Output:

Characteristics of Function Expression

- Functions are stored in variables.
- They may be anonymous (without a name).
- They are not fully hoisted.
- Cannot be called before initialization.

Example:

```
sayHello();  
  
const sayHello = function() {  
    console.log("Hello");  
};
```

This will produce an error because function expressions are not initialized during hoisting.

• Question 3: Discuss the concept of parameters and return values in functions.

Ans:

In JavaScript, parameters and return values are important concepts used in functions to make them flexible and reusable.

Parameters in Functions

Parameters are variables listed inside the parentheses of a function definition. They allow functions to receive input values when called.

Parameters make functions dynamic because different values can be passed each time the function is used.

Syntax of Parameters

```
function functionName(parameter1, parameter2) {  
    // code block  
}
```

Example of Parameters

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

```
greet("Aryan");
```

Output:

Hello Aryan

Explanation

- name is the parameter.
- "Aryan" is the argument passed during function call.
- The parameter receives the argument value.

Multiple Parameters Example

```
function add(a, b) {  
  console.log(a + b);  
}
```

```
add(10, 20);
```

Output:

30

Here:

- a and b are parameters.
- 10 and 20 are arguments.

Return Values in Functions

A return value is the value a function sends back after execution using the `return` keyword.

The `return` statement ends function execution and provides output that can be stored or used later.

Syntax of Return Statement

```
function functionName() {  
    return value;  
}
```

Example of Return Value

```
function square(num) {  
    return num * num;  
}
```

```
let result = square(5);
```

```
console.log(result);
```

Output:

25

Explanation

- The function calculates $5 * 5$.
- `return` sends the result back.
- The returned value is stored in the variable `result`.

Arrays

• Question 1: What is an array in JavaScript? How do you declare and initialize an array?

Ans:

In JavaScript, an array is a special type of object used to store multiple values in a single variable. Arrays allow developers to store collections of data such as numbers, strings, or objects together instead of creating separate variables for each value.

Arrays are very useful when working with lists of data, such as:

- student names

- product lists
- marks
- colors
- user information

Each value in an array is called an element, and every element has an index number. Array indexing starts from 0.

Features of Arrays

1. Stores multiple values in one variable
2. Elements are ordered
3. Supports different data types
4. Dynamic in size
5. Provides built-in methods for data manipulation

Declaring an Array

An array is declared using square brackets [].

Syntax

```
let arrayName = [];
```

Initializing an Array

Initializing means assigning values to the array during declaration.

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```

In this example:

- fruits is the array name.
- "Apple", "Banana", and "Mango" are elements.

• Question 2: Explain the methods push(), pop(), shift(), and unshift() used in arrays.

Ans:

In JavaScript, arrays provide several built-in methods for adding and removing elements. Among the most commonly used methods are:

- `push()`
- `pop()`
- `shift()`
- `unshift()`

These methods help modify arrays dynamically.

1. `push()` Method

The `push()` method is used to add one or more elements to the end of an array.

Syntax

```
arrayName.push(value);
```

Example

```
let fruits = ["Apple", "Banana"];
```

```
fruits.push("Mango");
```

```
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Mango"]
```

Explanation

- "Mango" is added at the end of the array.
- The original array size increases.

2. `pop()` Method

The `pop()` method removes the last element from an array.

Syntax

```
arrayName.pop();
```

Example

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.pop();

console.log(fruits);
```

Output:

```
["Apple", "Banana"]
```

Explanation

- The last element "Mango" is removed from the array.

3. shift() Method

The `shift()` method removes the first element from an array.

Syntax

```
arrayName.shift();
```

Example

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.shift();

console.log(fruits);
```

Output:

```
["Banana", "Mango"]
```

Explanation

- "Apple" is removed from the beginning of the array.

- Remaining elements shift left.

4. unshift() Method

The unshift() method adds one or more elements at the beginning of an array.

Syntax

```
arrayName.unshift(value);
```

Example

```
let fruits = ["Banana", "Mango"];
```

```
fruits.unshift("Apple");
```

```
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Mango"]
```

Explanation

- "Apple" is added at the beginning.
- Existing elements move to the right.

Objects

• Question 1: What is an object in JavaScript? How are objects different from arrays?

Ans:

In JavaScript, an object is a non-primitive data type used to store data in the form of key-value pairs. Objects are used to represent real-world entities and organize related data together in a structured way.

Each value stored inside an object is called a property, where:

- the **key** represents the property name
- the **value** represents the actual data

Objects are widely used in JavaScript because they allow developers to store complex and related information efficiently.

Creating an Object

Objects are created using curly braces {}.

Syntax

```
let objectName = {  
  key1: value1,  
  key2: value2  
};
```

Example of an Object

```
let student = {  
  name: "Aryan",  
  age: 21,  
  city: "Surat"  
};
```

In this example:

- name, age, and city are keys
- "Aryan", 21, and "Surat" are values

The object stores related information about a student in a single variable.

Features of Objects

1. Stores data in key-value form
2. Can hold multiple data types
3. Used to represent real-world entities
4. Dynamic and flexible
5. Supports functions and nested objects

Difference Between Objects and Arrays

Although both arrays and objects are used to store data, they work differently.

Feature	Object	Array
Data Storage	Key-value pairs	Ordered list of values
Syntax	{ }	[]
Access Method	Using property names	Using index numbers
Order Importance	Not necessary	Important
Use Case	Complex structured data	Collection of similar data

Example of Array

```
let fruits = ["Apple", "Banana", "Mango"];
```

Here:

- Elements are accessed using indexes like fruits[0].

Example of Object

```
let car = {
  brand: "Toyota",
  color: "Black"
};
```

Here:

- Values are accessed using property names like car.brand.

Explanation

- Arrays are best for storing lists of values.
- Objects are best for storing detailed information with meaningful property names.

For example:

- A list of marks → Array
- Student details → Object

Thus, objects provide better structure and readability when handling complex data in JavaScript.

- **Question 2: Explain how to access and update object properties using dot notation and bracket notation.**

Ans:

In JavaScript, object properties can be accessed and updated using two methods:

1. Dot notation
2. Bracket notation

Both methods are commonly used for working with objects.

1. Dot Notation

Dot notation uses a dot . followed by the property name to access or update object values.

Syntax

objectName.propertyName

Example of Accessing Properties

```
let student = {  
  name: "Aryan",  
  age: 21  
};
```

```
console.log(student.name);  
console.log(student.age);
```

Output:

```
Aryan  
21
```

Updating Properties Using Dot Notation

```
student.age = 22;
```

```
console.log(student.age);
```

Output:

22

Explanation

- `student.age = 22` changes the value of the age property.
- Dot notation is simple and easy to read.

2. Bracket Notation

Bracket notation uses square brackets `[]` with the property name written as a string.

Syntax

```
objectName["propertyName"]
```

Example of Accessing Properties

```
let student = {  
  name: "Aryan",  
  age: 21  
};  
  
console.log(student["name"]);  
console.log(student["age"]);
```

Output:

Aryan
21

Updating Properties Using Bracket Notation

```
student["age"] = 23;  
  
console.log(student["age"]);
```

Output:

23

When Bracket Notation is Useful

Bracket notation is useful when:

- property names contain spaces
- property names are stored in variables
- dynamic property access is needed

JavaScript Events

- **Question 1: What are JavaScript events? Explain the role of event listeners.**

Ans:

In JavaScript, events are actions or occurrences that happen in the browser and can be detected by JavaScript. Events allow web pages to respond to user interactions and browser activities, making websites interactive and dynamic.

An event can occur when:

- a user clicks a button
- a key is pressed
- a form is submitted
- the mouse moves
- a webpage loads
- an image is hovered over

JavaScript listens for these events and performs specific actions when they happen.

Event Listeners in JavaScript

An event listener is a JavaScript feature used to detect events and execute a function automatically when the event occurs.

Event listeners help separate HTML structure from JavaScript behavior, making code cleaner and easier to maintain.

Role of Event Listeners

Event listeners are important because they:

1. Detect user interactions
2. Execute functions when events occur
3. Improve interactivity of web pages
4. Allow multiple events on elements
5. Keep JavaScript code organized

• Question 2: How does the `addEventListener()` method work in JavaScript? Provide an example.

Ans:

In JavaScript, the `addEventListener()` method is used to attach an event listener to an HTML element. It allows JavaScript to listen for specific events and execute a function when those events occur.

The `addEventListener()` method is one of the most commonly used methods for handling events in modern JavaScript.

Syntax of `addEventListener()`

```
element.addEventListener(event, function);
```

Parameters of `addEventListener()`

Parameter	Description
event	Type of event such as click or mouseover

Parameter	Description
function	Function to execute when event occurs

How addEventListener() Works

1. JavaScript selects an HTML element.
2. addEventListener() attaches an event to that element.
3. The browser waits for the event.
4. When the event occurs, the connected function executes.

Example of addEventListener()

```
<!DOCTYPE html>
<html>
<body>

<button id="btn">Click Me</button>

<script>

let button = document.getElementById("btn");

button.addEventListener("click", function() {
    alert("Button Clicked");
});

</script>

</body>
</html>
```

Explanation

In this example:

- document.getElementById("btn") selects the button.
- addEventListener("click", function(){}) attaches a click event.
- When the button is clicked, the function executes.
- The alert message appears on the screen.

Advantages of addEventListener()

1. Keeps HTML and JavaScript separate
2. Allows multiple events on the same element

3. Improves code readability
4. More flexible than inline events
5. Commonly used in modern web development

DOM Manipulation

• Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

Ans:

In JavaScript, the DOM (Document Object Model) is a programming interface that represents an HTML document as a tree-like structure of objects. It allows JavaScript to access, modify, create, and delete HTML elements dynamically.

When a webpage loads in the browser, the browser converts the HTML document into a structured model called the DOM. Every HTML element becomes an object inside this structure.

The DOM acts as a bridge between:

- HTML structure
- CSS styling
- JavaScript functionality

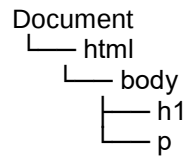
Structure of the DOM

The DOM organizes webpage elements in a hierarchical tree structure.

Example HTML:

```
<html>
  <body>
    <h1>Hello</h1>
    <p>Welcome</p>
  </body>
</html>
```

DOM Structure:



Each element is treated as a node/object that JavaScript can manipulate.

Role of the DOM

The DOM allows JavaScript to:

1. Access HTML elements
2. Change content dynamically
3. Modify CSS styles
4. Handle user events
5. Create or remove elements
6. Build interactive webpages

Without the DOM, JavaScript would not be able to interact with webpage content.

How JavaScript Interacts with the DOM

JavaScript interacts with the DOM by selecting HTML elements and performing operations on them.

JavaScript can:

- change text
- update styles
- add attributes
- remove elements
- respond to events

Example of DOM Manipulation

```
<!DOCTYPE html>
<html>
<body>
```

```
<h1 id="title">Hello World</h1>
```

```
<script>
```

```
document.getElementById("title").innerHTML = "Welcome Aryan";
```

```
</script>
```

```
</body>
```

```
</html>
```

Explanation

- document represents the entire webpage.
- getElementById("title") selects the <h1> element.
- innerHTML changes the content of the element.

Output Before JavaScript:

Hello World

Output After JavaScript:

Welcome Aryan

• Question 2: Explain the methods getElementById(), getElementsByClassName(), and querySelector() used to select elements from the DOM.

Ans:

In JavaScript, JavaScript provides several methods to select HTML elements from the DOM. These methods allow developers to access and manipulate webpage content dynamically.

The most commonly used methods are:

1. getElementById()
2. getElementsByClassName()
3. querySelector()

1. getElementById()

The getElementById() method selects a single HTML element using its unique id attribute.

Syntax

```
document.getElementById("idName");
```

Example

```
<h1 id="heading">Hello</h1>
```

```
<script>
```

```
let element = document.getElementById("heading");
```

```
console.log(element);
```

```
</script>
```

Explanation

- "heading" is the id of the element.
- JavaScript selects the element with that id.
- IDs should be unique within a webpage.

Common Uses

- Changing text
- Updating styles
- Handling events

2. getElementsByClassName()

The `getElementsByClassName()` method selects all elements having the same class name.

It returns an HTML collection containing multiple elements.

Syntax

```
document.getElementsByClassName("className");
```

Example

```
<p class="text">First</p>
```

```
<p class="text">Second</p>
```

```
<script>
```

```
let elements = document.getElementsByClassName("text");
```

```
console.log(elements);  
  
</script>
```

Explanation

- Both <p> elements have the class "text".
- JavaScript returns a collection of matching elements.

Accessing Individual Elements

```
console.log(elements[0]);
```

This accesses the first element in the collection.

3. `querySelector()`

The `querySelector()` method selects the first element that matches a specified CSS selector.

It is one of the most flexible DOM selection methods.

Syntax

```
document.querySelector("selector");
```

Example Using ID

```
<h1 id="title">Welcome</h1>
```

```
<script>
```

```
let element = document.querySelector("#title");
```

```
console.log(element);
```

```
</script>
```

Example Using Class

```
<p class="info">Hello</p>
```

```
<script>
```

```
let element = document.querySelector(".info");
```

```
console.log(element);
```

</script>

Explanation

- # represents an ID selector.
- . represents a class selector.
- querySelector() returns only the first matching element.

JavaScript Timing Events (setTimeout, setInterval)

- **Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?**

Ans:

In JavaScript, setTimeout() and setInterval() are built-in timing functions used to execute code after a specific delay or repeatedly at fixed intervals. These functions are part of JavaScript's asynchronous programming features and are commonly used for handling timed events in web applications.

Timing functions help developers:

- delay actions
- create animations
- show notifications
- update content automatically
- perform repeated tasks

1. setTimeout() Function

The `setTimeout()` function is used to execute a function or block of code once after a specified time delay.

Syntax of setTimeout()

```
setTimeout(function, time);
```

Parameters

Parameter	Description
function	Function to execute
time	Delay time in milliseconds

Note:

- 1000 milliseconds = 1 second

How setTimeout() Works

1. JavaScript starts the timer.
2. The program continues executing other code.
3. After the specified delay, the function executes once.

Example

```
setTimeout(function() {  
    console.log("Hello");  
}, 3000);
```

Output After 3 Seconds:

Hello

Uses of setTimeout()

- Showing popup messages
- Delayed notifications
- Redirecting webpages
- Animation effects
- Waiting before executing tasks

2. setInterval() Function

The setInterval() function repeatedly executes a function after a fixed time interval.

Syntax of setInterval()

```
setInterval(function, time);
```

Parameters

Parameter	Description
function	Function to execute repeatedly
time	Interval time in milliseconds

How setInterval() Works

1. JavaScript starts the timer.
2. The function executes repeatedly after every specified interval.
3. The process continues until stopped manually.

Example

```
setInterval(function() {  
    console.log("Running...");  
}, 2000);
```

Output Every 2 Seconds:

```
Running...  
Running...  
Running...
```

Uses of setInterval()

- Digital clocks
- Live timers
- Auto-refreshing data
- Slideshows
- Real-time updates

Stopping setInterval()

The clearInterval() method is used to stop a repeating interval.

Example:

```
let timer = setInterval(function() {  
    console.log("Hello");  
}, 1000);  
  
clearInterval(timer);
```

Timing Events in JavaScript

Timing events are actions executed after a delay or at repeated intervals using timing functions.

JavaScript uses:

- setTimeout() for one-time delayed execution
- setInterval() for repeated execution

These functions make web applications interactive and dynamic.

• Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.

Ans:

In JavaScript, the setTimeout() function can be used to delay the execution of a function by a specified amount of time.

Since:

1000 milliseconds = 1 second

A delay of 2 seconds means:

2000 milliseconds

Example of setTimeout() with 2-Second Delay

```
setTimeout(function() {  
    console.log("This message appears after 2 seconds");  
}, 2000);
```

JavaScript Error Handling

**• Question 1: What is error handling in JavaScript?
Explain the try, catch, and finally blocks with an example.**

Ans:

In JavaScript, error handling is a process used to detect, manage, and respond to runtime errors that occur during program execution. Errors can happen for many reasons such as invalid input, incorrect code, network issues, or unexpected conditions.

If errors are not handled properly, the JavaScript program may stop execution and create a poor user experience. Error handling helps developers manage these problems gracefully without crashing the application.

JavaScript mainly uses:

- try
- catch
- finally

blocks for handling errors.

try Block

The try block contains the code that may produce an error. JavaScript first executes the code inside the try block.

Syntax

```
try {  
    // code that may cause error  
}
```

catch Block

The catch block executes when an error occurs inside the try block. It handles the error and prevents the program from stopping completely.

Syntax

```
catch(error) {  
    // code to handle error  
}
```

The error parameter stores information about the error.

finally Block

The finally block contains code that always executes whether an error occurs or not.

Syntax

```
finally {  
    // always executes  
}
```

The finally block is usually used for:

- cleanup operations
- closing connections
- stopping loaders
- releasing resources

Complete Example of try, catch, and finally

```
try {  
  
    let result = 10 / 0;
```

```
    console.log(result);  
  } catch(error) {  
    console.log("An error occurred");  
  } finally {  
    console.log("Execution completed");  
  }  
}
```

Output

Infinity
Execution completed

• Question 2: Why is error handling important in JavaScript applications?

Ans:

In JavaScript, error handling is extremely important because JavaScript applications often run in dynamic and unpredictable environments such as web browsers and servers. Errors can occur at any time due to invalid user input, coding mistakes, network failures, or unexpected conditions.

Without proper error handling, even a small error can stop the execution of the entire application and create a poor user experience.

Importance of Error Handling

1. Prevents Program Crashes

Error handling prevents JavaScript applications from stopping suddenly when an error occurs. Instead of crashing, the program can continue running safely.

2. Improves User Experience

Users should not see broken pages or unexpected failures. Error handling allows developers to display meaningful messages such as:

- "Something went wrong"
- "Please try again later"

This makes applications more user-friendly.

3. Helps in Debugging

Error messages help developers identify and fix issues quickly during development and maintenance.

Example:

```
console.log(error.message);
```

This provides details about the problem.

4. Ensures Application Stability

Modern web applications perform many tasks such as:

- API requests
- form submissions
- file uploads
- database communication

Error handling ensures these operations work reliably even if problems occur.

5. Handles Unexpected Situations

Applications may face unexpected situations such as:

- internet disconnection
- missing data
- invalid calculations
- server errors

Error handling allows developers to manage these situations properly.

6. Improves Security

Proper error handling prevents sensitive technical details from being exposed to users. Instead of displaying system information, developers can show safe error messages.

Real-Life Example

In an online shopping website:

- if payment processing fails,
- error handling can display:

Payment failed. Please try again.

instead of crashing the entire webpage.